

IBMTechSales / liberty-enterprise-deployment-labs Public

<> Code Issues Pull requests Actions Projects Security and quality

liberty-enterprise-deployment-labs  
/ 1030-Value-of-Liberty-dynamic-  
routing  
/ README.md

kpostreich fix typo in 1030 lab

d8431e1 · 2 years ago

Preview Code Blame

Raw Copy Download Edit

# Lab 1030: The value of Dynamic Routing with Liberty



Last updated: March 2023

Duration: 45 minutes

Need support? Contact Kevin Postreich, Yi Tang

## Introduction

In the modern-day business, the application is king. To provide workload balancing and failover protection for application high availability, the WebSphere Plug-in can be used with an Apache web server to route HTTP requests to the application running in application servers.

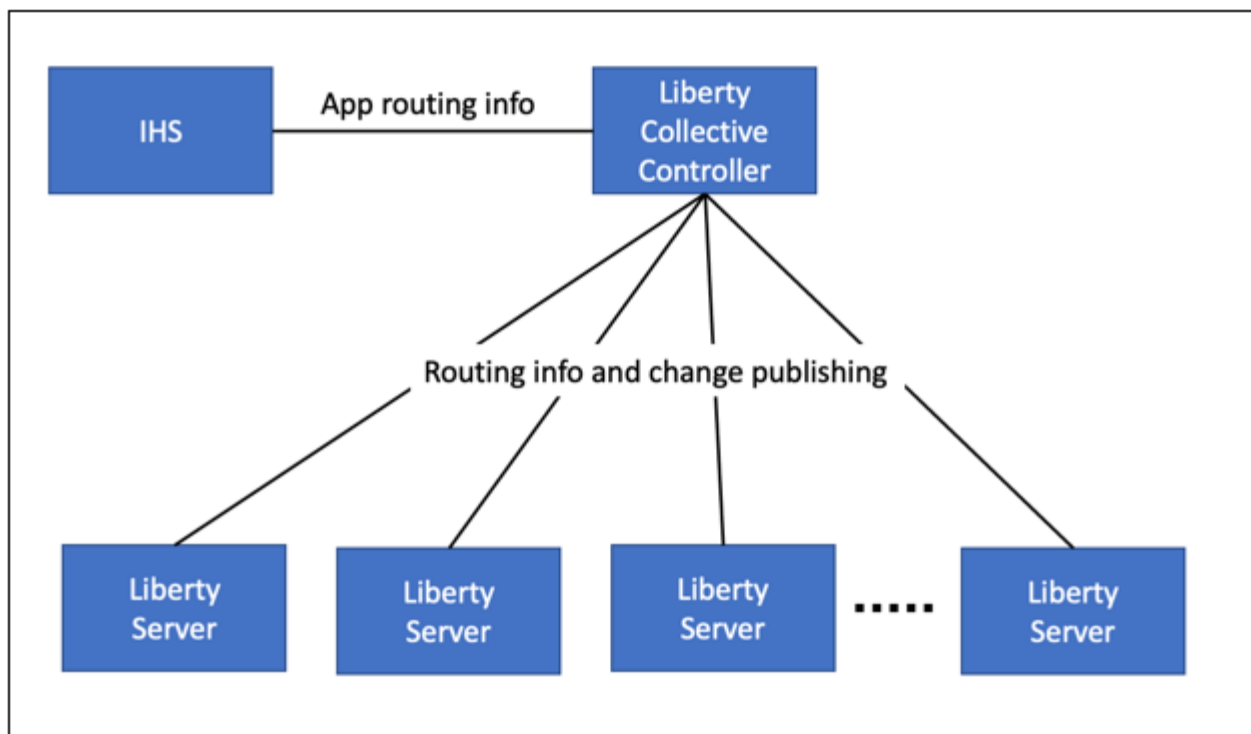
Traditionally this is done by creating the plug-in configuration for each application server and using a utility to merge these configurations into a single file, then copying it to the web server installation.

The Liberty dynamic routing feature enables routing of HTTP requests to members of Liberty collectives without regenerating the WebSphere plug-in configuration file when the environment changes.

When servers, collective members, applications, or virtual hosts are added, removed, started, stopped, or modified; the new information is dynamically delivered to the WebSphere plug-in through the Liberty Collective Controller.

Requests are routed based on up-to-date information. In this approach, the web server plug-in configuration file (plugin-cfg.xml) only needs to contain routing information about the collective controller process(es).

The plug-in then contacts the controller to obtain information about all the servers in the collective and directs HTTP requests to the appropriate Liberty servers in the collective.



In this Lab, you will learn how to setup and use Liberty dynamic routing capability to provide high availability for your applications.

## Accessing the environment

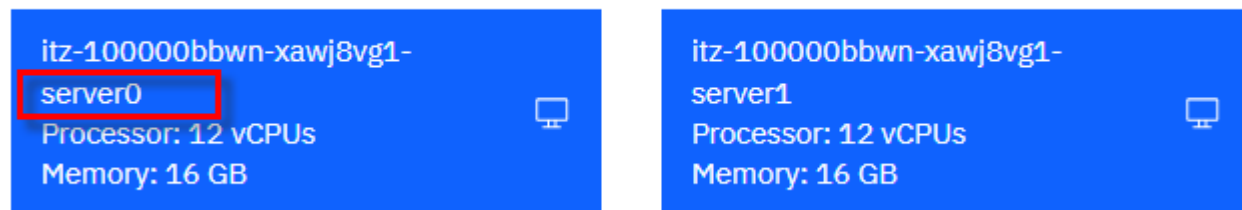
If you are doing this lab as part of an instructor led workshop (virtual or face to face), an environment has already been provisioned for you. The instructor will provide the details for accessing the lab environment.

Otherwise, you will need to reserve an environment for the lab. You can obtain one here. Follow the on-screen instructions for the "**Reserve now**" option.

KLP: TBD LINK TO ENV RESERVATION

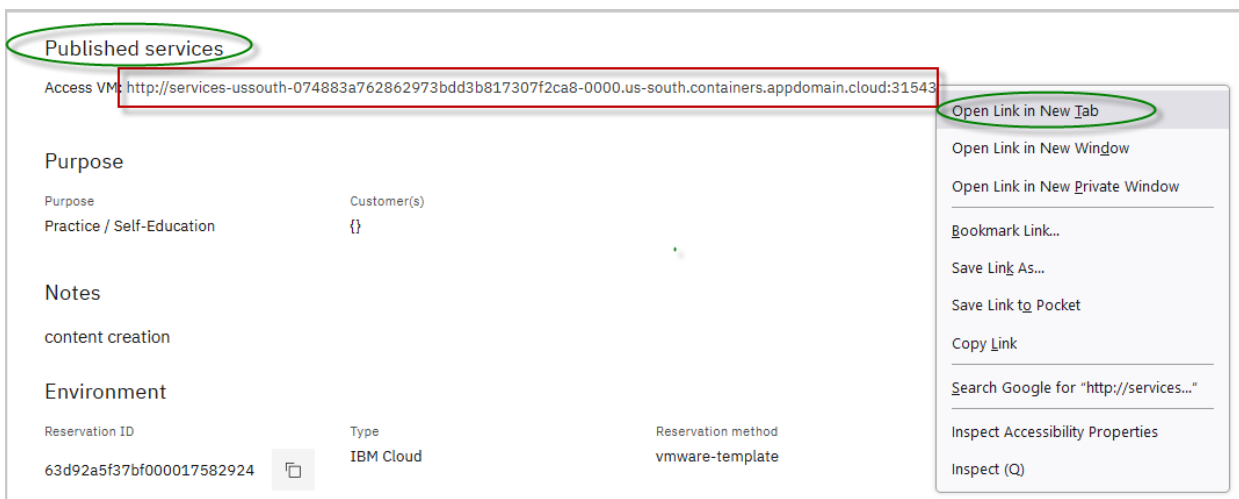
The lab environment contains two (2) Linux VMs.

### VM Remote Console



A Published Service is configured to provide access to the **server0** VM through the noVNC interface for the lab environment.

1. Access the lab environment from your web browser.
  - a. When the environment is provisioned, right-mouse click on the **Published Service** link/ Then select "**Open link in New Tab**" from the context menu.



Published services

Access VM <http://services-ussouth-074883a762862973bdd3b817307f2ca8-0000.us-south.containers.appdomain.cloud:31543>

Purpose

Purpose Practice / Self-Education Customer(s) {}

Notes

content creation

Environment

Reservation ID 63d92a5f37bf00017582924 Type IBM Cloud Reservation method vmware-template

Open Link in New Tab

Open Link in New Window

Open Link in New Private Window

Bookmark Link...

Save Link As...

Save Link to Pocket

Copy Link

Search Google for "http://services..."

Inspect Accessibility Properties

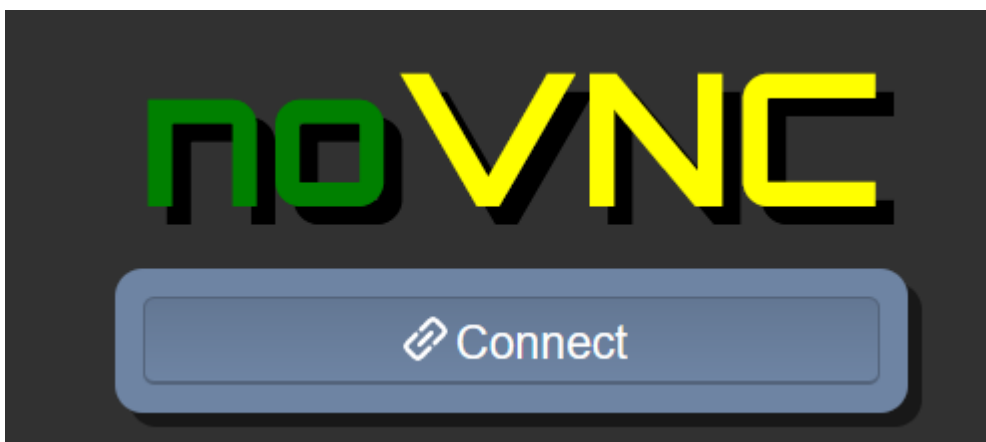
Inspect (Q)

b. Click on the "vnc.html" link to open the lab environment through the noVNC interface.

## Directory listing for /

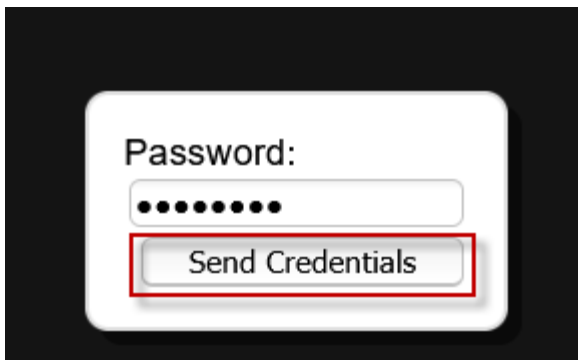
- [app/](#)
- [core/](#)
- [include/](#)
- [utils/](#)
- [vendor/](#)
- [vnc.html](#)
- [vnc\\_auto.html@](#)
- [vnc\\_lite.html](#)

c. Click the **Connect** button



d. Enter the password as: **passw0rd**. Then click the **Send Credentials** button to access the lab environment.

**Note:** That is a numeric zero in passw0rd



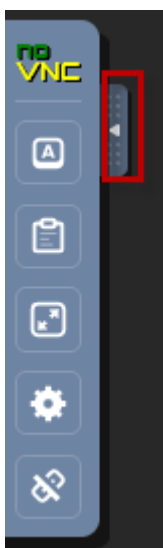
2. Login to the **server0** VM using the credentials below:

- User ID: **techzone**
- Password: **IBMDem0s!**

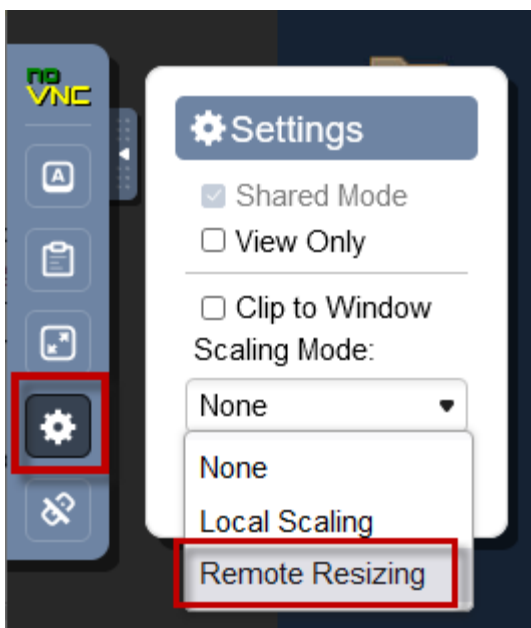
## Tips for working in the lab environment

1. You can resize the viewable area using the **noVNC Settings** options to resize the virtual desktop to fit your screen.

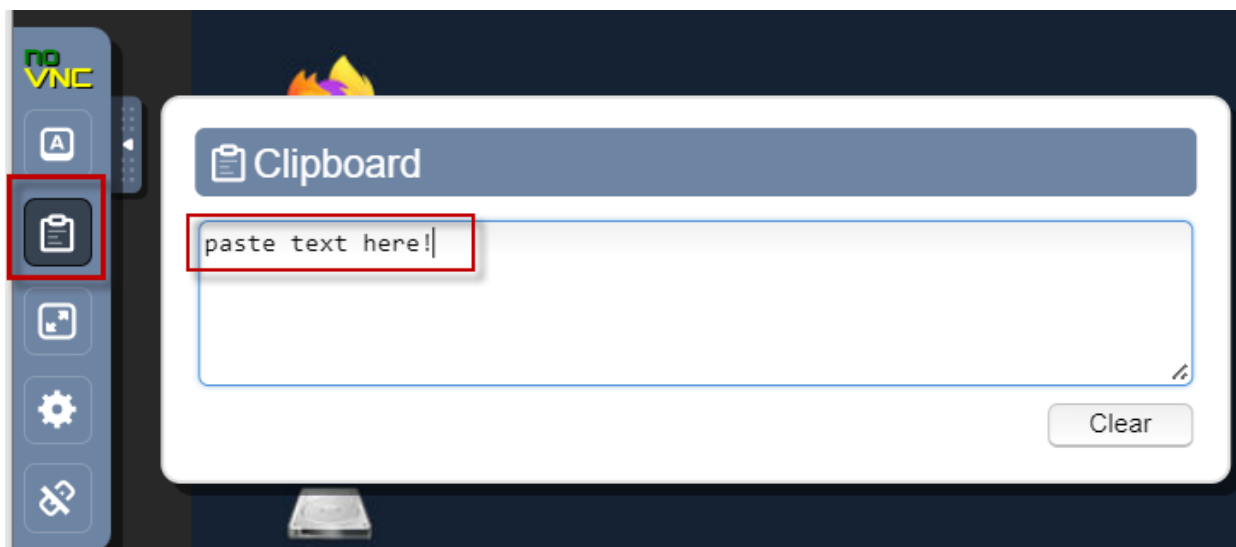
a. From the environemnt VM, click on the **twisty** on the noNC control pane to open the menu.



b. To increase the visible area, click on **Settings > Scaling Mode** and set the value to **Remote Resizing**



2. You can copy / paste text from the lab guide into the lab environment using the clipboard in the noVNC viewer.
  - a. Copy the text from the lab guide that you want to paste into the lab environment
  - b. Click the **Clipboard** icon and **paste** the text into the noVNC clipboard



- c. Paste the text into the VM, such as to a terminal window, browser window, etc.
  - d. Click on the **clipboard** icon again to close the clipboard

**NOTE:** Sometimes pasting into a Terminal window in the VM does not work consistently.

In this case you might try again, or open another Terminal Window and try again, or paste the text into a **Text Editor** in the VM, and then paste it into the Terminal window in the VM.

3. An alternative to using the noVNC Copy / Paste option, you may consider opening the lab guide in a web browser inside of the VM. Using this method, you can easily copy / paste text from the lab guide without having to use the noVNC clipboard.

## Lab: The value of Dynamic Routing with Liberty

For the Dynamic Routing function to work, the **dynamicRouting-1.0** feature is required only for the **collective controllers**.

There is no requirement to upgrade every collective member. Dynamic Routing can be used to route to applications installed in any Liberty server in a collective, including:

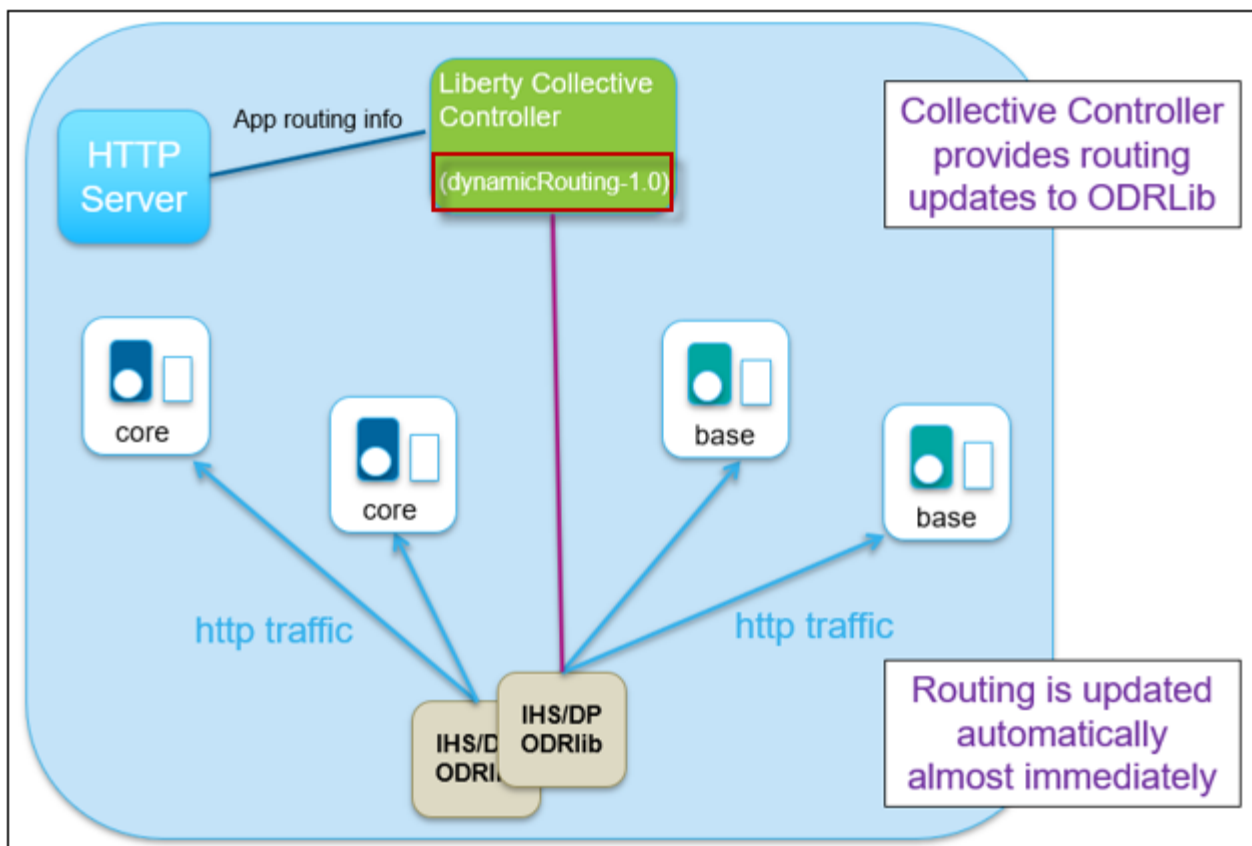
- Liberty Application Server Network Deployment
- Liberty Application Server Liberty Base edition
- Liberty Application Server Core



### Tip:

The ability to leverage Liberty's dynamic routing capability for any edition of Liberty is powerful and cost effective.

You get request HA for Liberty Base and Core editions, without requiring Liberty ND licenses. It also eliminates time-consuming plug-in merges and manual updates to the HTTP server plugin configuration if servers are added or removed from the configuration.



## Clone the GitHub repo for this workshop

This lab requires artifacts that are stored in a GitHub repository. Run the command below to clone the repository to the local VM used for the lab.

1. If not already done so in a previous lab, clone the GitHub repo that contains lab artifacts needed for the lab.
  - a. Open a new terminal window on the "server0.gym.lan" VM



- b. Clone the GitHub repository required for the lab

```
git clone https://github.com/IBMTechSales/liberty_admin_pot.git
```



c. Navigate to the “**lab-scripts**” directory in the cloned repo

```
cd ~/liberty_admin_pot/lab-scripts
```



d. Add the “**execute**” permissions to the lab-scripts directories and shell scripts

```
chmod -R 755 ./
```



## Part 1: Verify the HTTP server

---

In this lab, you generate a special web server plug-in configuration file that provides dynamic routing information to allow a web server to spray HTTP requests across two liberty servers that are running the two sample applications, **PlantsByWebSphere** and **WhereAml**.

You will start and stop members in the collective to experience the dynamic routing behavior.

**This lab contains the following activities:**

- Verify the HTTP server
- Create a Liberty Collective (**If you did not complete Lab\_1020**)
- Configure Dynamic Routing
- Start Liberty Member Servers from Liberty Admin Center
- Test the Dynamic Routing Features
- Configure and Test Dynamic Routing Rules
- Summary

A **web server** and **WebSphere Plug-in** (Plug-in) are two important components in this workload balancing and failover protection solution. You must have a web server that is supported by the web server plug-in for WebSphere Application Server.

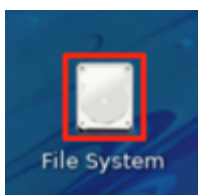
In this lab the web server used is **IBM HTTP Server** (IHS).

For the simplicity of the lab, IHS and the Plug-in have been pre-installed and configured on the **server0.gym.lan** VM in the following directory locations and HTTP port.

- The installation directory of IHS is **/opt/IBM/HTTPServer**
- The installation directory for Plug-in is **/opt/IBM/WebSphere/Plugins**.
- IHS is configured to listen on secure port **8443**

In this section, you ensure the IBM HTTP Server starts and runs as expected

1. Double-click **File System** icon on the desktop to open the file explorer

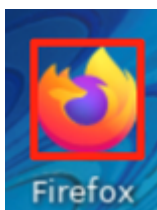


2. From a new terminal window run command to start **IHS** server

```
/opt/IBM/HTTPServer/bin/apachectl start
```



3. Open a web browser window by double-clicking the **Firefox** icon on the desktop.



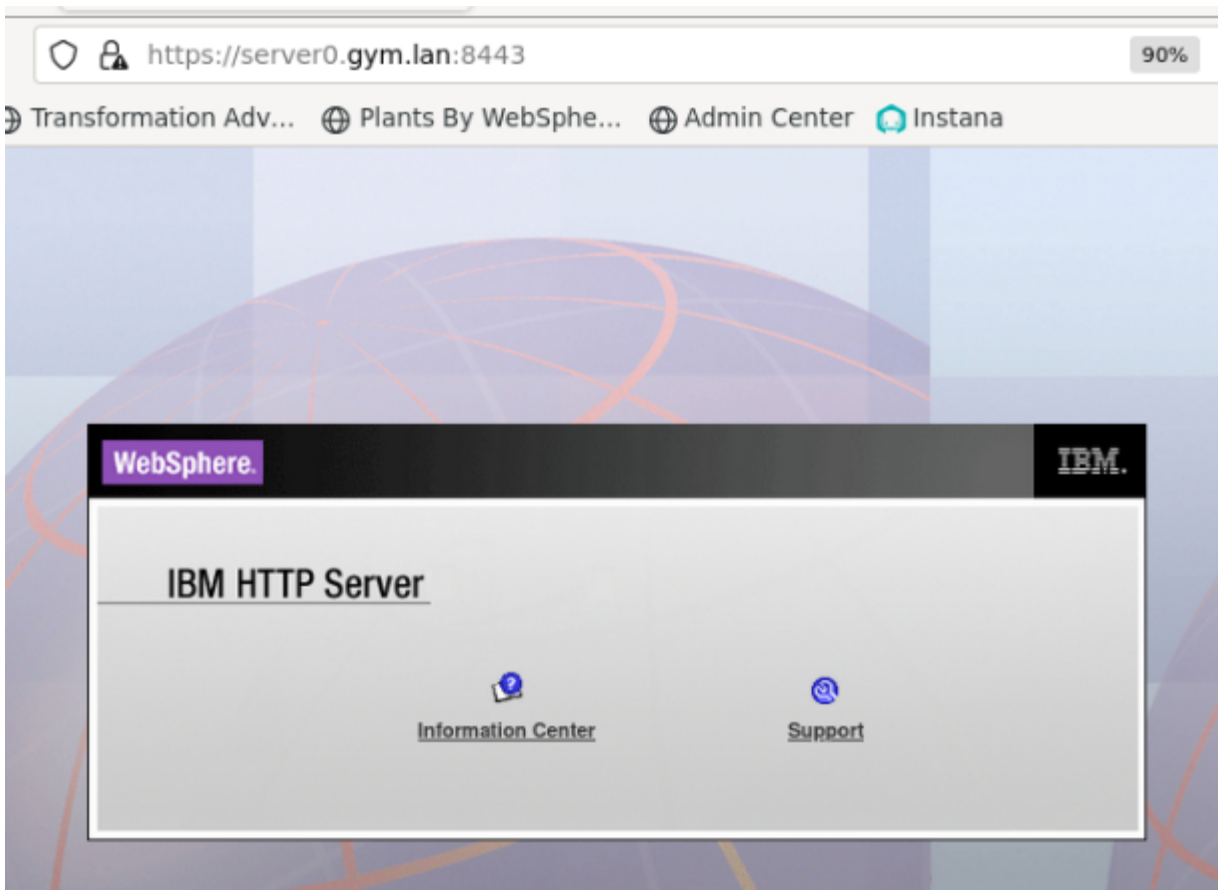
4. From the browser, go to the **IHS** URL:

```
https://server0.gym.lan:8443
```



**Note:** The HTTP server is running on secure port **8443**, not the default secure port 443

The **IHS** home page is displayed.



5. Run command below to **stop** the **IHS** server

```
/opt/IBM/HTTPServer/bin/apachectl stop
```



## Part 2: Ensure the Liberty Collective is deployed

Liberty dynamic routing requires a Liberty Collective, and the application servers which host the applications used in the labs to be members of the Liberty collective. The learning module for creating the Liberty collective is "Lab\_1020".

In this section you will ensure that a Liberty administrative collective is available, and the application servers are deployed to the collective.



### IMPORTANT: Please read!

If you have completed the **Lab 1020** of this series, you have already created the Liberty collective controller.

In other words, you can skip **Part 2** of the lab and continue with **Part 3** if you have already completed Lab 1020 in this series.

The Admin Center ULR is: <https://server0.gym.lan:9491/adminCenter>

The Admin Center credentials are: **admin / admin**

## 2.1: If you have not completed lab 1020, the following steps provide a "Fast path" to creating the Liberty collective required for this lab, "Lab 1030".

1. Run the command below to ensure a Liberty collective is created:

```
/home/techzone/liberty_admin_pot/lab-scripts/deployCollective.sh  
--lab1030
```



The deployCollective.sh script will do the following:

- If the Controller already exists, the script will ABORT, as a collective already exists
- If the Controller does NOT exist, it will be created
- Build and produce a Liberty Server package to deploy to Liberty application servers
- Create two Liberty Servers, "appServer1" and "appServer2", deploy the server package, and join the servers to the Liberty Collective

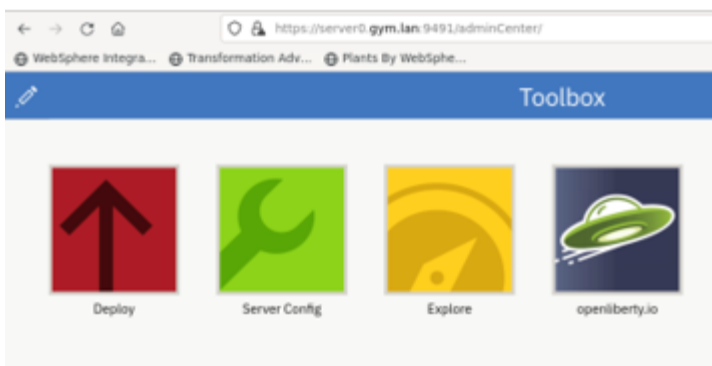
2. Once the script completes, access the **Admin Center**. Enter the login credentials as:  
**admin / admin**

```
https://server0.gym.lan:9491/adminCenter
```

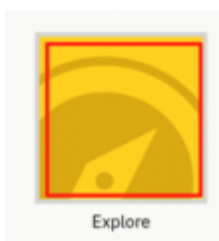


**Note:** If you see the "Warning: Potential Security Risk Ahead", click **Advanced..->Accept Risk and Continue** to continue.

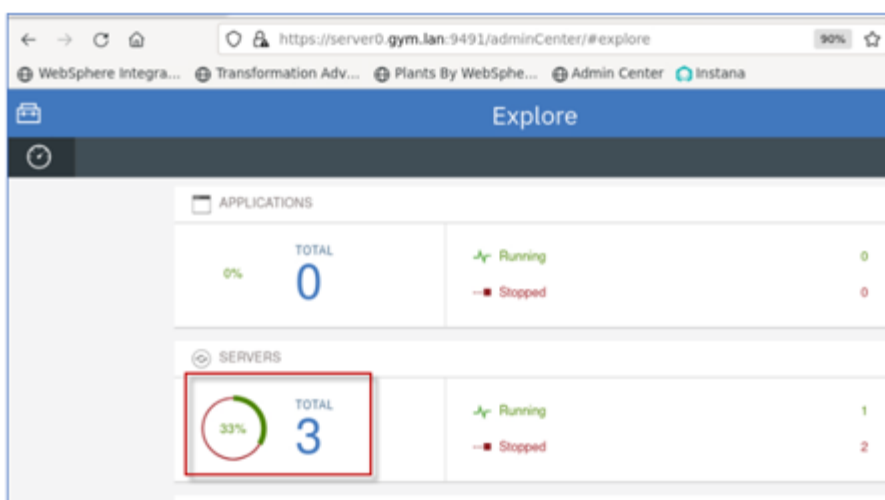
The Liberty collective Admin Center page is displayed.



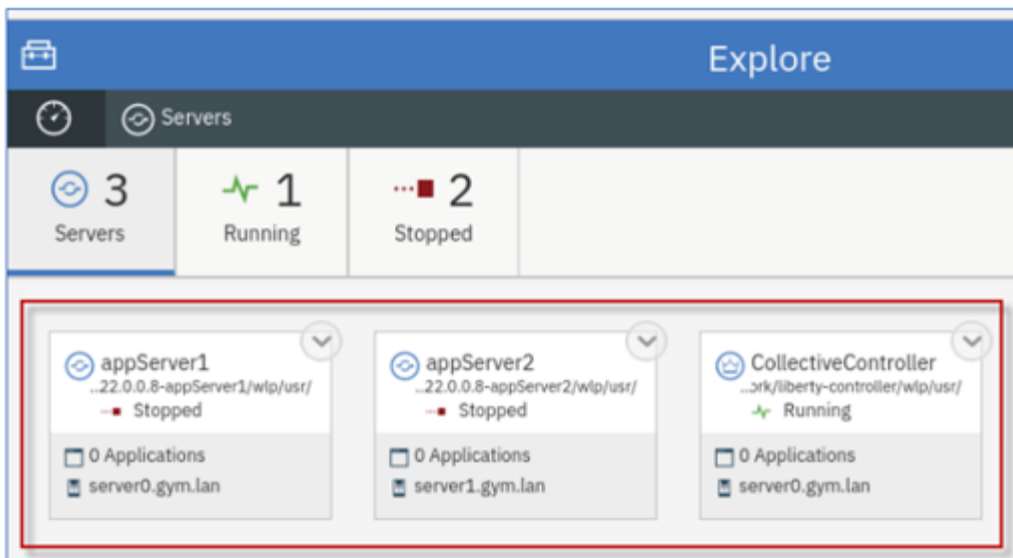
3. Click the **Explore** icon



The collective resource list is displayed, and you can see that you have three servers.



4. Click the Servers list to see the three servers, appServer1, appServer2, and CollectiveController



## AVOID TROUBLE!

If you ran the "**deployCollective.sh --skip1030**" command, and it ended with the following information, as illustrated in the screen shot below, it means that the script detected the Collective Controller has already been created.

Therefore, the scripts existed and displayed the URL to the Admin Center.

Go to the Admin Center URL and verify that the collective contains the two collective members, "appServer1" and "appServer2" as illustrated above.

## Troubleshooting!

Admin Center app cannot run, or the two application servers are not part of the collective, then manual cleanup of the collective is required, and you should contact the lab instructor.

```
=====
Running 'createController.sh'
=====
--force NOT specified, but the Controller seems to already be created.

Server CollectiveController is running with process ID 25234.
rc: 0
  Exiting!

-----
The Collective Controller already exists.
-> Admin Center URL: https://server0.gym.lan:9491/adminCenter
-----

# End of createController.sh script.
Return code from createController script: 12

-----
-> The Collective Controller already exists. Exiting deployCollective script!
-> You can access the Liberty Admin Center using the URL displayed above.
-----

[techzone@server0 lab-scripts]$
```

## Part 3: Configure Dynamic Routing Feature

In this section, you configure the Dynamic Routing feature to route HTTP requests to members of Liberty collectives without having to regenerate the WebSphere plug-in configuration file when the environment changes.

The Dynamic Routing feature, "**dynamicRouting-1.0**", provides the Dynamic Routing service, which dynamically retrieves routing information from the collective repository and delivers this information to the WebSphere plug-in.

To configure dynamic routing for a Liberty collective, you need to perform the following tasks:

- **Add dynamicRouting-1.0 feature to the Collective controller**

This feature must be added to the Collective Controller's server.xml file.

- **Create a Plug-in configuration file for the HTTP Server**

The "dynamicRouting setup" command generates the "keystore" and "plug-in configuration files" required for dynamic routing.

- **Establish a secure connection between the plug-in and the collective controller**

The generated plug-in configuration file and keys must be copied to the appropriate locations to establish the secure connection.

The high-level steps to configure Dynamic Routing are listed below, as described in the IBM documentation:



**Tip:**

In this lab, you are provided an **automation script** that performs ALL the following steps for you!

- Enable Dynamic Routing in the controller by adding the dynamicRouting-1.0 feature to the collective controller server.xml file
- Ensure the Controller server is started, once dynamic routing is enabled
- Run the "**dynamic routing setup**" command from the controller server, which generates the keystore and plug-in configuration files
- Copy the generated plugin-key.p12 and plugin-cfg.xml files to a temporary directory on the web server host
- Convert the keystore to Certificate Management System (CMS) format, which is the supported format for the HTTP Server
- Copy the generated plug-in files to the IHS server
- Start the HTTP Server

Additional details for setting up Dynamic Routing for a collective and running the "dynamicRouting" command can be found in the IBM Documentation links below:

**IBM Documentation - Setting up Dynamic Routing for a Collective:**

[https://www.ibm.com/docs/en/was-liberty/nd?topic=SSAW57\\_liberty/com.ibm.websphere.wlp.zseries.doc/ae/twlp\\_wve\\_enabledynrout\\_single.htm](https://www.ibm.com/docs/en/was-liberty/nd?topic=SSAW57_liberty/com.ibm.websphere.wlp.zseries.doc/ae/twlp_wve_enabledynrout_single.htm)

**IBM Documentation – Dynamic Routing Command:**

[https://www.ibm.com/docs/en/was-liberty/nd?topic=SSAW57\\_liberty/com.ibm.websphere.wlp.zseries.doc/ae/rwlp\\_wve\\_dynroutcollect.htm](https://www.ibm.com/docs/en/was-liberty/nd?topic=SSAW57_liberty/com.ibm.websphere.wlp.zseries.doc/ae/rwlp_wve_dynroutcollect.htm)

## Setup Dynamic Routing

In this section, you will use an automation script, which we provide in the lab environment, to perform the steps described above to setup and configure Dynamic Routing.

1. Run the `setupDynamicRouting.sh` script shown below to setup the plugin configuration for dynamic routing.

The **`setupDynamicRouting.sh`** script performs all the tasks described above which configures dynamic routing in the collective

```
/home/techzone/liberty_admin_pot/lab-scripts/  
setupDynamicRouting.sh
```



Once the command is completed, the plug-in configuration files are created and configured for the IHS server.

```
dynamicRouting setup completed  
gskcmd convert completed  
[NOTE: This command is deprecated]  
  
gskcmd cert completed  
[techzone@server0 ~]$
```

**Dynamic routing in the Liberty Collective is now ready to use!**

## Examine the generated “plugin-cfg.xml” file

The **`plugin-cfg.xml`** file contains configuration information that determines how the web server plug-in forwards requests to the Liberty servers in the collective.

The plugin only needs to connect to the Collective Controller to get topology information. It does not need to know the host/port of the application servers.

The `plugin-cfg.xml` file is in the following directory:

```
/opt/IBM/WebSphere/Plugins/config/webserver1
```

1. Examine the generated **`plugin-cfg.xml`**

```
gedit /opt/IBM/WebSphere/Plugins/config/webserver1/plugin-cfg.xml
```



With Dynamic Routing, HTTP requests are sent to members of Liberty collectives without regenerating the WebSphere plug-in configuration file when the environment changes.

**Note:**

The plugin-cfg.xml no does not contain the host and port information for the application servers.

Instead, the plugin-cfg.xml contains the host and port information for the collective controller which provides the application and application server information dynamically to the plugin.

When servers, cluster members, applications, or virtual hosts are added, removed, started, stopped, or modified; the new information is dynamically delivered to the WebSphere plug-in from the Liberty Collective Controller.

In this configuration, requests are routed based on up-to-date information.

```
1 <?xml version="1.0" encoding="UTF-8"?><!-- HTTP server plugin config file for webserver1 generated on
2023.03.03 at 10:22:29 EST--><!--Merged HTTP server plugin config file--><Config
ASDDisableNagle="false" AcceptAllContent="false" AppServerPortPreference="HostHeader"
ChunkedResponse="false" ConfigHash="477462759" FIPSEnable="false" IISDisableNagle="false"
IISPluginPriority="High" IgnoreDNSFailures="false" RefreshInterval="60" ResponseChunkSize="64"
SSLConsolidate="false" TrustedProxyEnable="false" VHostMatchingCompat="false">
2   <Log LogLevel="Error" Name="/opt/IBM/WebSphere/Plugins/logs/webserver1/http_plugin.log"/>
3   <Property Name="ESIEnable" Value="true"/>
4   <Property Name="ESIMaxCacheSize" Value="1024"/>
5   <Property Name="ESIInvalidationMonitor" Value="false"/>
6   <Property Name="ESIEnableToPassCookies" Value="false"/>
7   <Property Name="PluginInstallRoot" Value="/opt/IBM/WebSphere/Plugins/">
8 <!-- Configuration generated using httpEndpointRef=defaultHttpEndpoint-->
9 <!-- The default_host contained only aliases for endpoint defaultHttpEndpoint.
10     The generated VirtualHostGroup will contain only configured web server ports:
11         webserverPort=80
12         webserverSecurePort=443 -->
13
14
15
16
17 <Property Name="Keyfile" Value="/opt/IBM/WebSphere/Plugins/config/webserver1/plugin-key.kdb"/>
18 <Property Name="Stashfile" Value="/opt/IBM/WebSphere/Plugins/config/webserver1/plugin-key.sth"/>
19 <IntelligentManagement>
20 <Property name="webserverName" value="webserver1"/>
21 <ConnectorCluster enabled="true" maxRetries="-1" name="defaultCollective" retryInterval="60">
22 <Property name="uri" value="/ibm/api/dynamicRouting"/>
23 <Connector host="server0.gym.lan" port="9491" protocol="https">
24 <Property name="keyring" value="/opt/IBM/WebSphere/Plugins/config/webserver1/plugin-key.kdb"/>
25 </Connector>
26 </ConnectorCluster>
27 <Property name="RoutingRulesConnectorClusterName" value="defaultCollective"/>
28 </IntelligentManagement>
29 </Config>
```

2. Close the gedit editor. DO NOT SAVE ANY CHANGES!

## Examine the Web Server's "httpd.conf" file

The httpd.conf file contains the HTTP Server configuration.

The WebSphere plug-in module is loaded by appending configuration to the `httpd.conf` file in the web server.

The web server's `httpd.conf` file is in the following directory:

`/opt/IBM/HTTPServer/conf`

1. Examine the generated **httpd.conf** file

```
gedit /opt/IBM/HTTPServer/conf/httpd.conf
```



- Scroll to the last line of the `httpd.conf` file, which is the configuration to load the WebSphere plugin module.
- Notice the configuration points to the **plugin-cfg.xml** file, which is used to determine how to direct the http requests to the Liberty servers in the collective.

```
860 |
861 |
862 LoadModule was_ap24_module /opt/IBM/WebSphere/Plugins/bin/64bits/mod_was_ap24_http.so
863 WebSpherePluginConfig /opt/IBM/WebSphere/Plugins/config/webserver1/plugin-cfg.xml
```

2. Close the **gedit** editor. DO NOT SAVE ANY CHANGES!

## Part 4: Start Liberty collective member servers

At this point, you should have two Liberty collective members should now be created in the collective, and named as follows:

- `appServer1`
- `appServer2`

In this section, you will **start** these two servers from the Liberty Admin Center if they are not already started.

The PlantsByWebSphere application requires an application database, which you need to ensure is up and running.



**Info:**

You may have already started the db2 container in a preceding lab.

To find out if the db2 container is already running, execute the following command:

```
docker ps | grep db2_demo_data
```

**FYI:** It is OK to execute the docker start command below, even if the container is already running.

1. Before starting the Liberty servers, you need to start the db2 database used by the **PlantsByWebSphere** application with the command below.

```
docker start db2_demo_data
```



2. Access the Liberty Admin Center from the browser

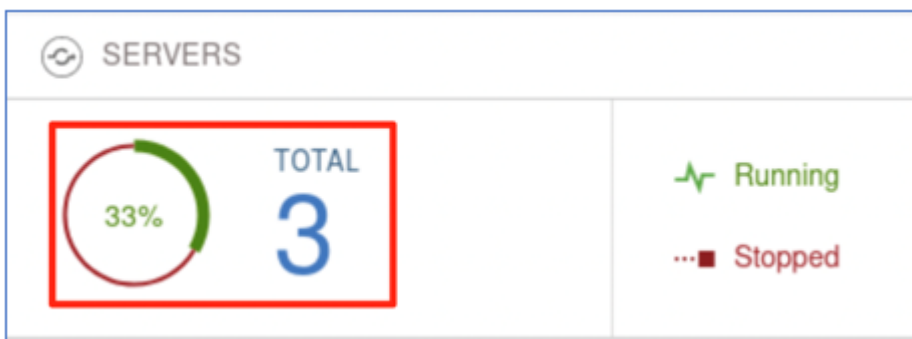
- a. Enter the login credentials as: admin / admin

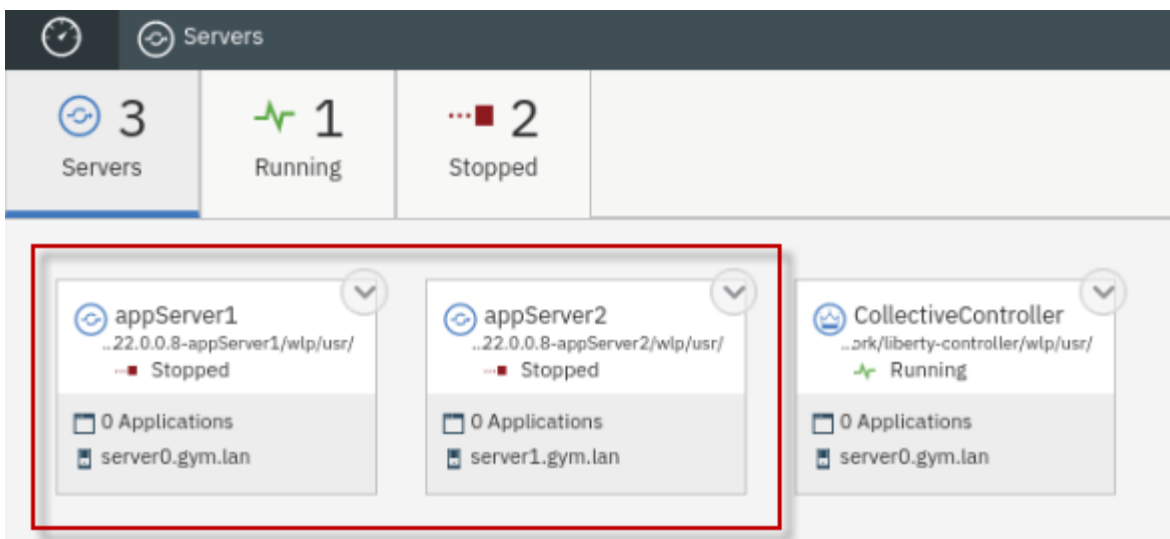
```
https://server0.gym.lan:9491/adminCenter
```



3. Go to the Servers view in the Admin Center

- a. From the Admin Center, go to the **Explorer** page, click the **SERVERS** icon to go to its details page.





### IMPORTANT: Please read!

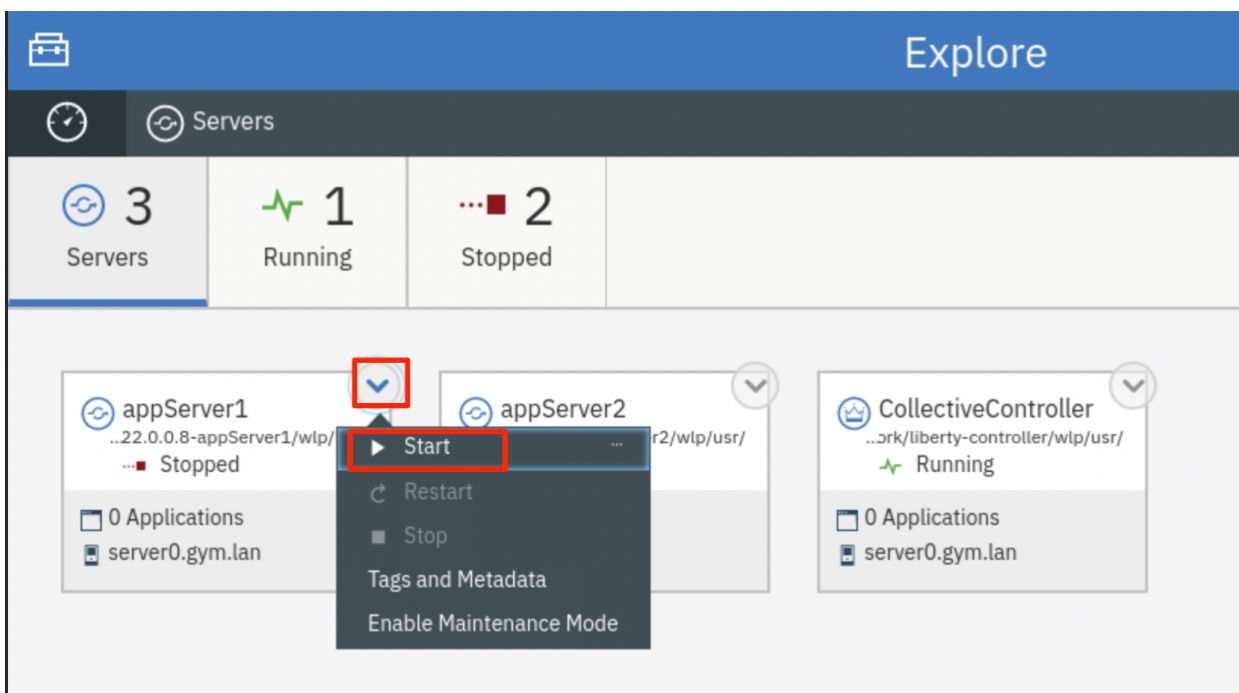
If the two servers illustrated above are NOT in your collective, then you cannot continue with the lab.

Ensure that you completed **Lab 1020** of this series, or you have completed the **Part 2** of this lab.

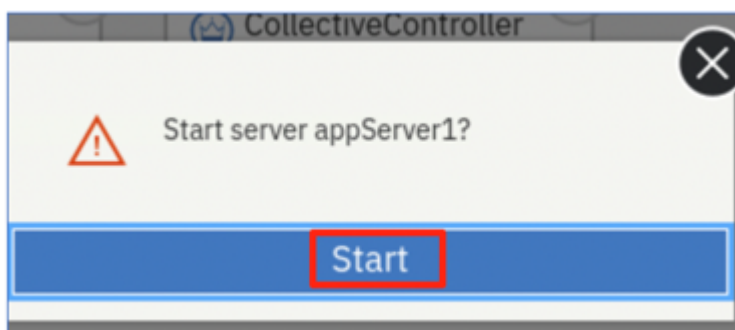
If you already completed Lab 1020, then you don't need to perform Part 4, if the two Liberty servers are in the "running" state.

#### 4. Start appServer1

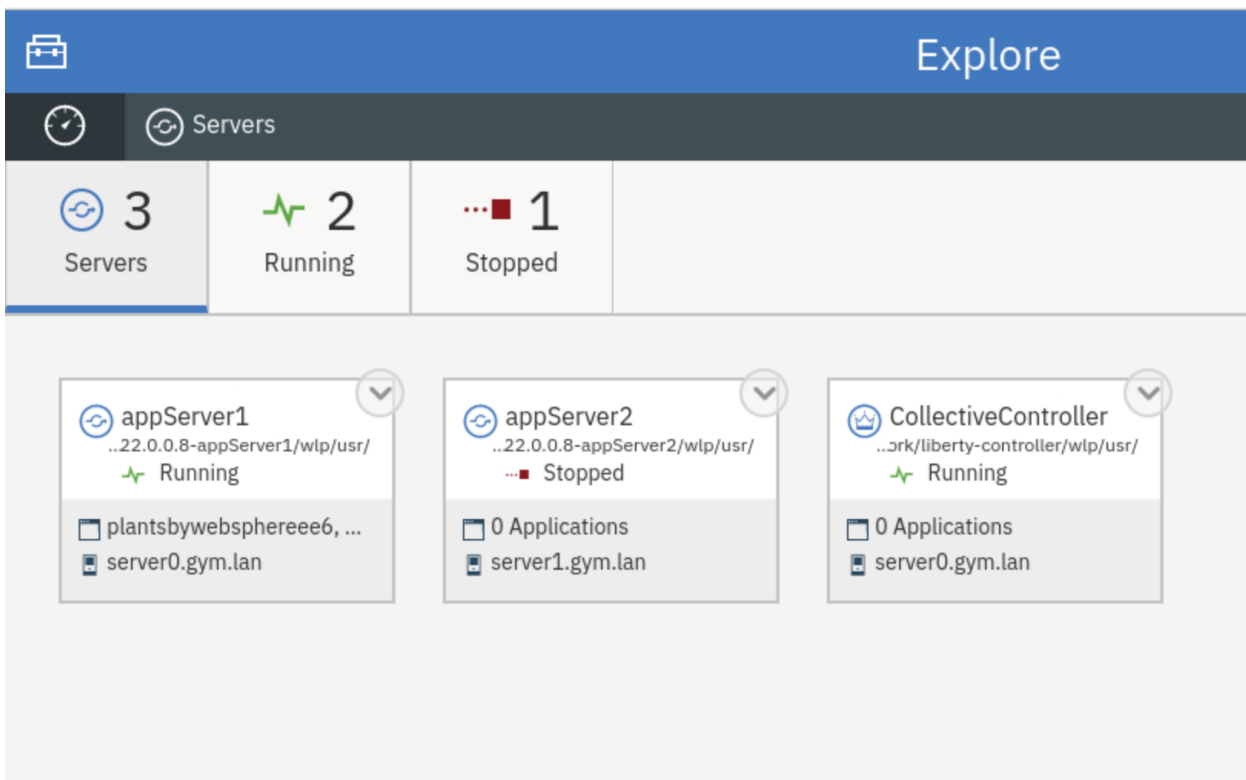
- In the server details page, click the dropdown menu icon of **appServer1** and select **Start** to start the server.



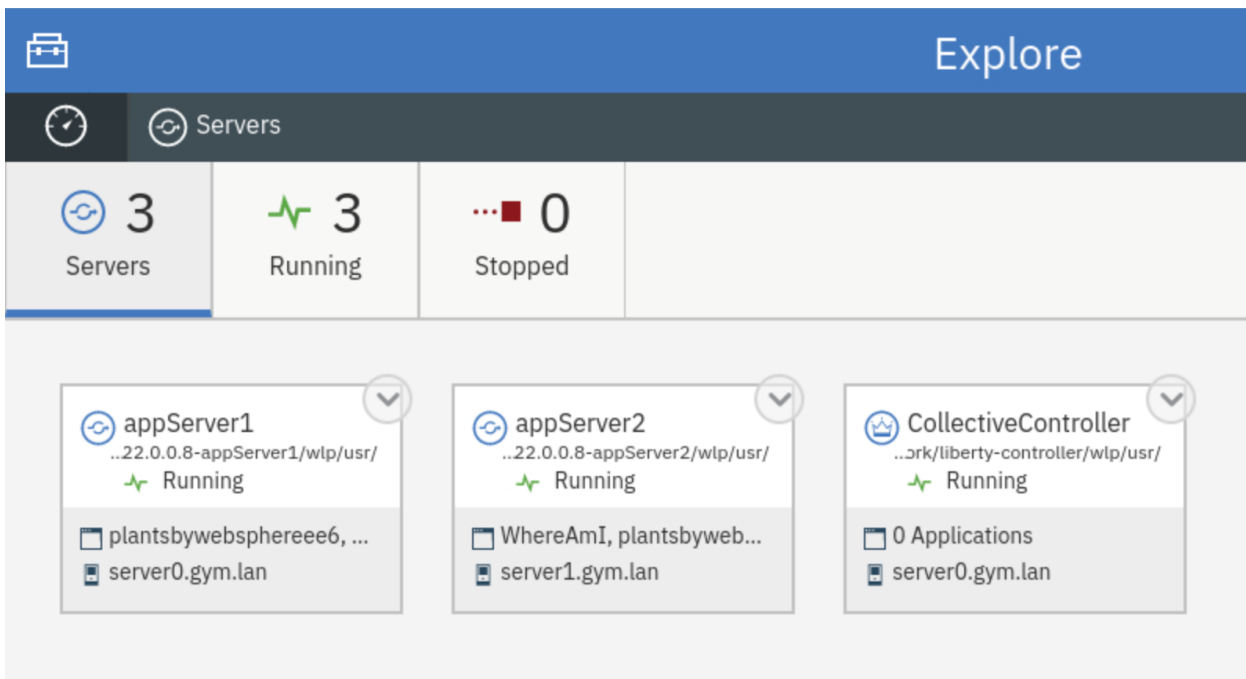
b. Click **Start** to confirm the start **appServer1** server command.



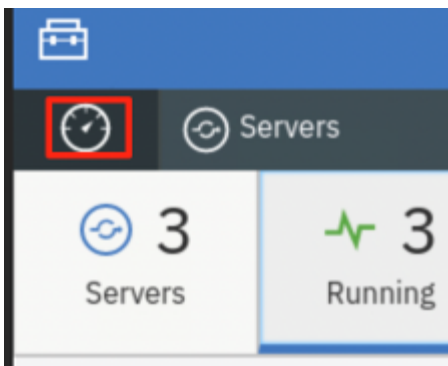
Server **appServer1** will get started, and you can see it is "**Running**" status with 2 applications.



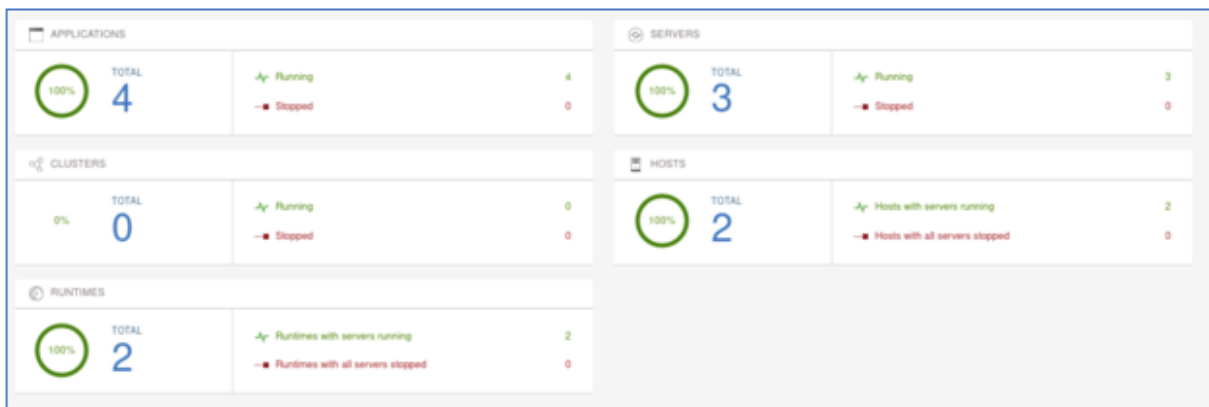
5. Repeat the same server start procedure for **appServer2** server. Once it is done, the **appServer2** server is started as show below:



6. Click the **Explorer** dashboard icon to go back to the dashboard view.



You will see all the two applications on both member servers are up and running.



## Part 5: Testing the Dynamic Routing Features

In this section, you are going to test the dynamic routing you configured for the Liberty collective.

You are going to conduct two testing scenarios:

- In the first test case, you are using the **PlantsByWebSphere** application to test the high availability of the application and verify you can always access the application directly from the IHS server if at least one of the member servers is running.

When you stop one of the app servers, the dynamic routing automatically redirects the traffic to another surviving app server without any user intervention or application interruption.

- The second test case demonstrates round robin load balancing and the dynamic routing distributes traffic to the collective members based on their workloads.

The **WhereAml** application is used in this testing because it does NOT use sticky sessions, whereas the **PlantsByWebSphere** application does.

When you refresh the application URL link in the web browser window, you can see the dynamic routing performs round-robin style routing among the servers.

## Test Case 1:

This test case uses PlantsByWebSphere application. The design of this application uses HTTP Sessions to store application state in the internal http session object. By default, the http session object is local to the Liberty server, and not persisted in any external store.

WebSphere traditional and WebSphere Liberty use a JSESSIONID in this case, which identifies the server handling the request that includes uses an http session. Then on subsequent transactions or requests, the JSESSIONID is read by the web server plugin, and requests continue to be routed to the SAME server.

If the server handling the requests goes down, then the web server plugin will redirect the requests to any surviving servers.

However, without session persistence configured, any session data is lost, such as items in a shopping cart, or login cookies, etc.

1. To access the **PlantsByWebSphere** application through IHS server and plugin, open a new browser window and enter the application URL as:

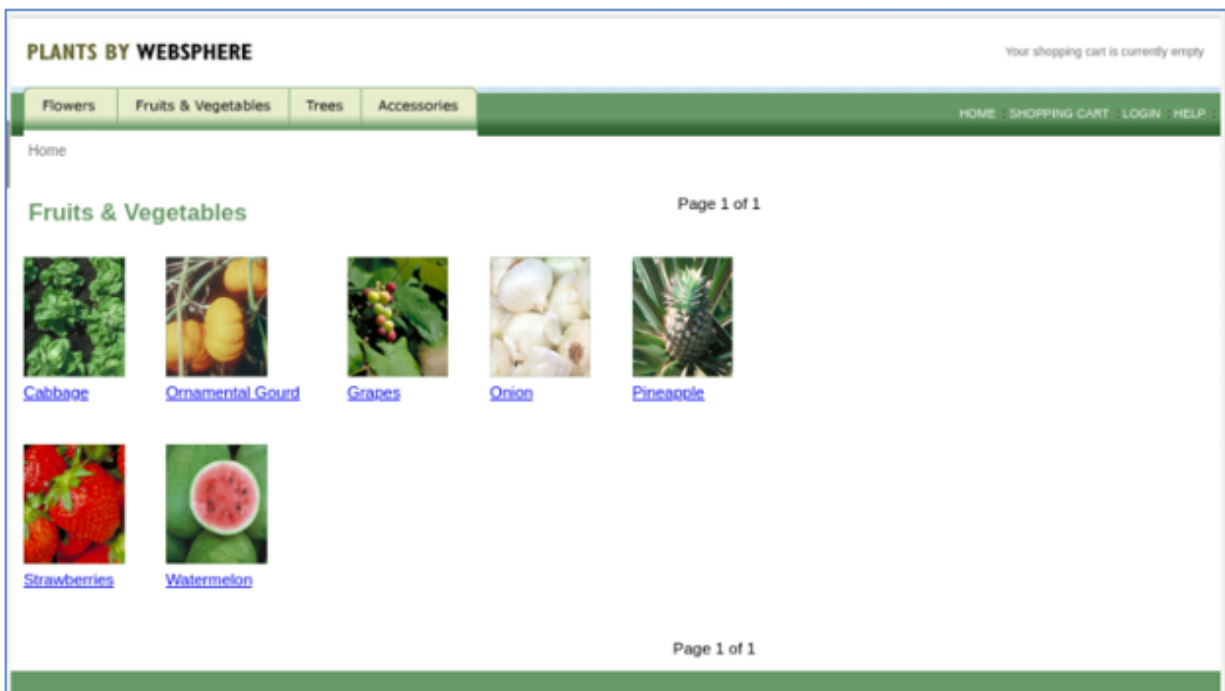
```
https://server0.gym.lan:8443/PlantsByWebSphere
```



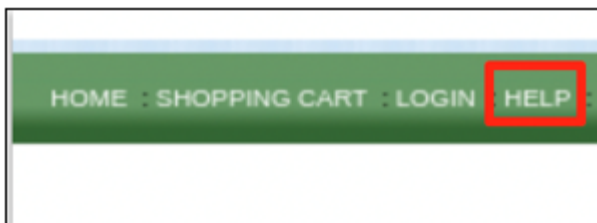
The application "**Home**" page is displayed.



- You can navigate and visit different pages of the application. You can see that although the application is running on two Liberty servers with different HTTP/HTTPS ports, the dynamic routing function of the Liberty collective is able to direct the incoming traffic through the specified IHS server port (8080) to the application.

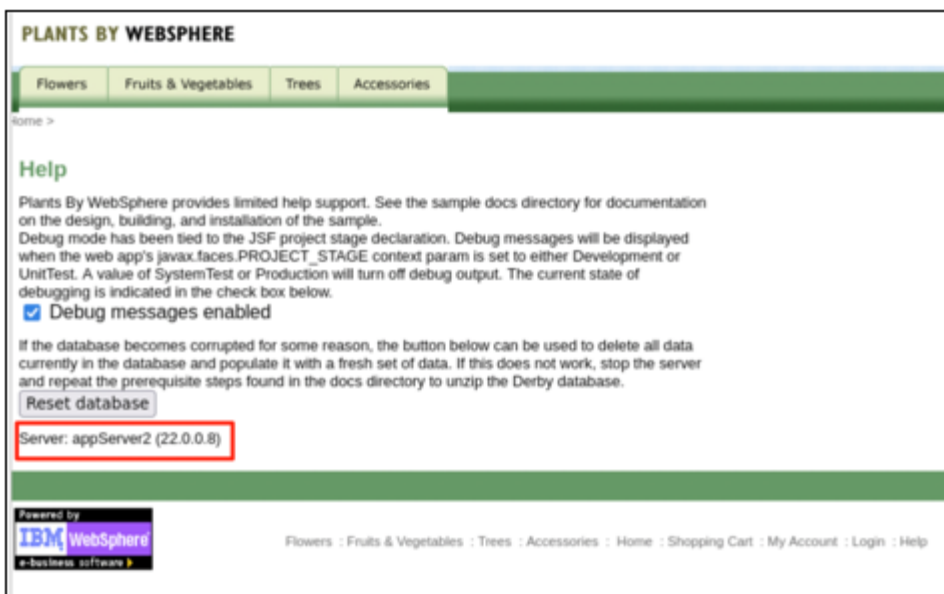


- Click the **Help** link to go to application **Help** page.



The application Help page is displayed. On this page, you can see which Liberty server the request was routed.

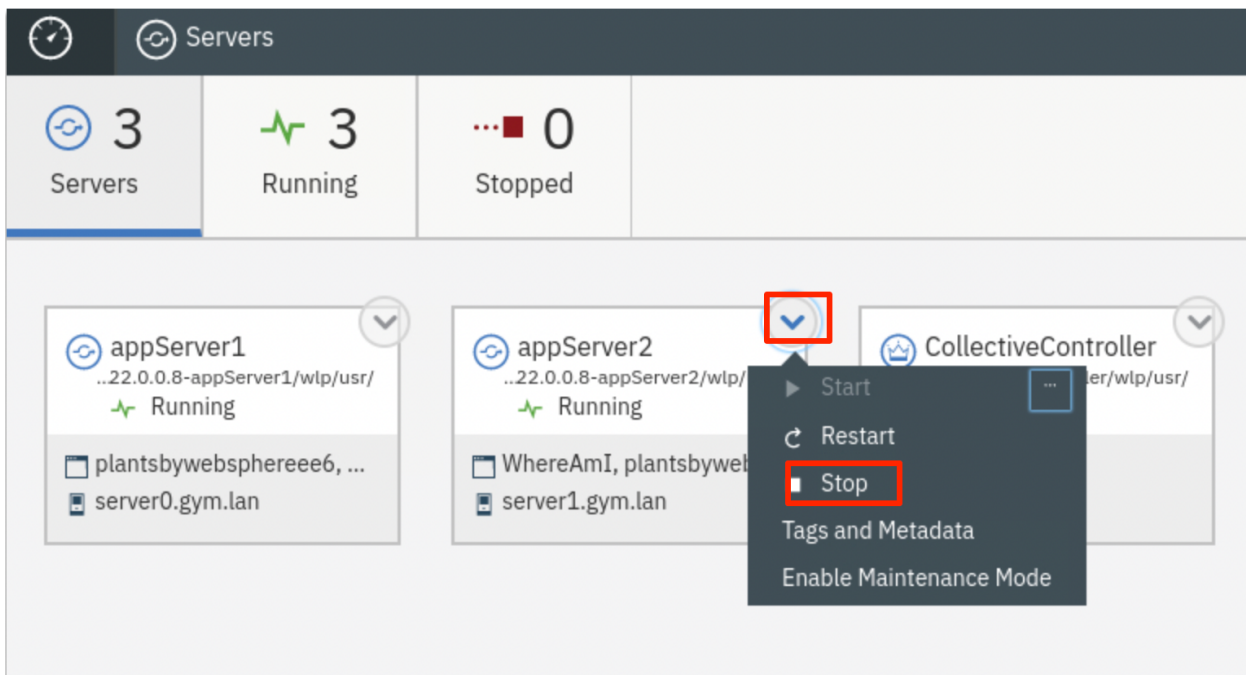
As showing in the screen shot below, the application is running from **appServer2** which might be different in your case.



4. **Stop** the Liberty server that is identified as handling the request, as shown on the PlantsByWebSphere application "**Help**" page.

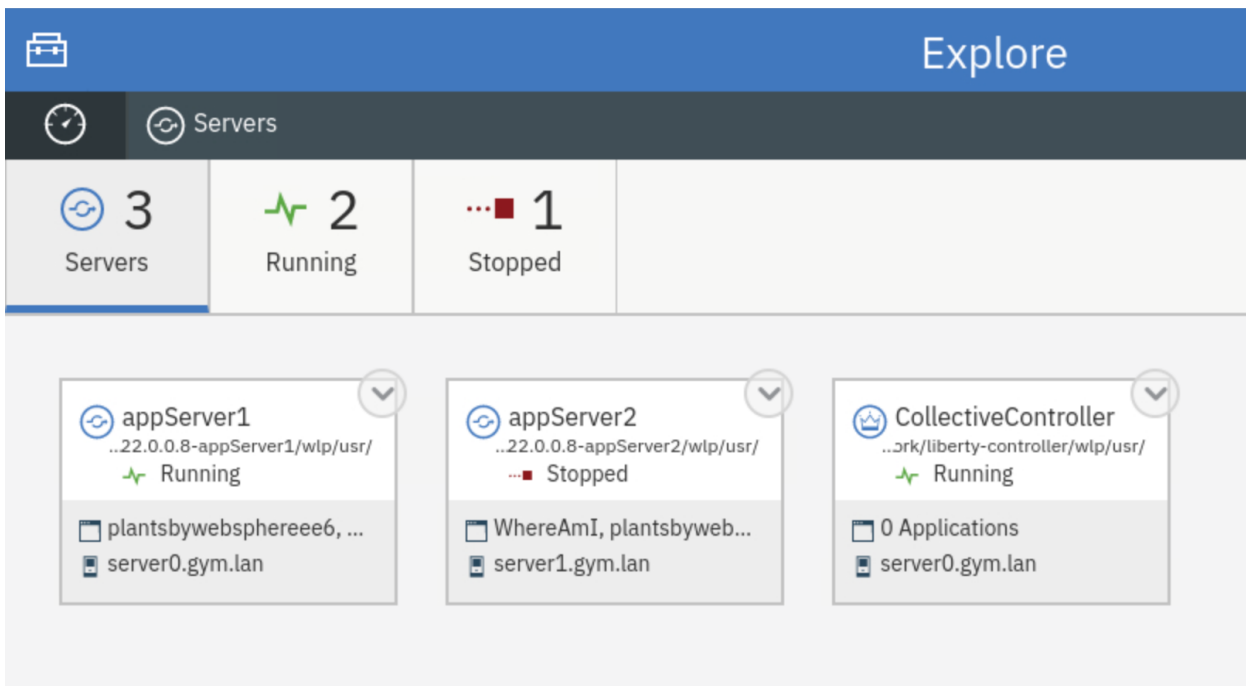
a. Go back to Liberty collective **Admin Center Servers** page

b. **Stop** the server that was identified on the application Help page, as illustrated below:

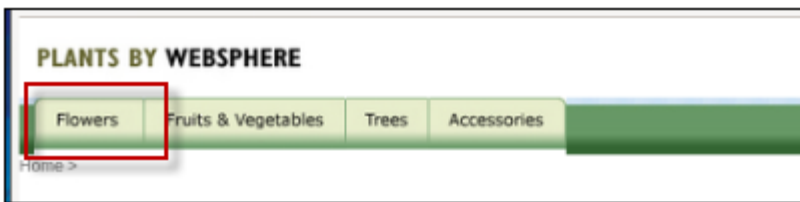


If prompted, enter the Admin Center credentials as: **admin / admin**.

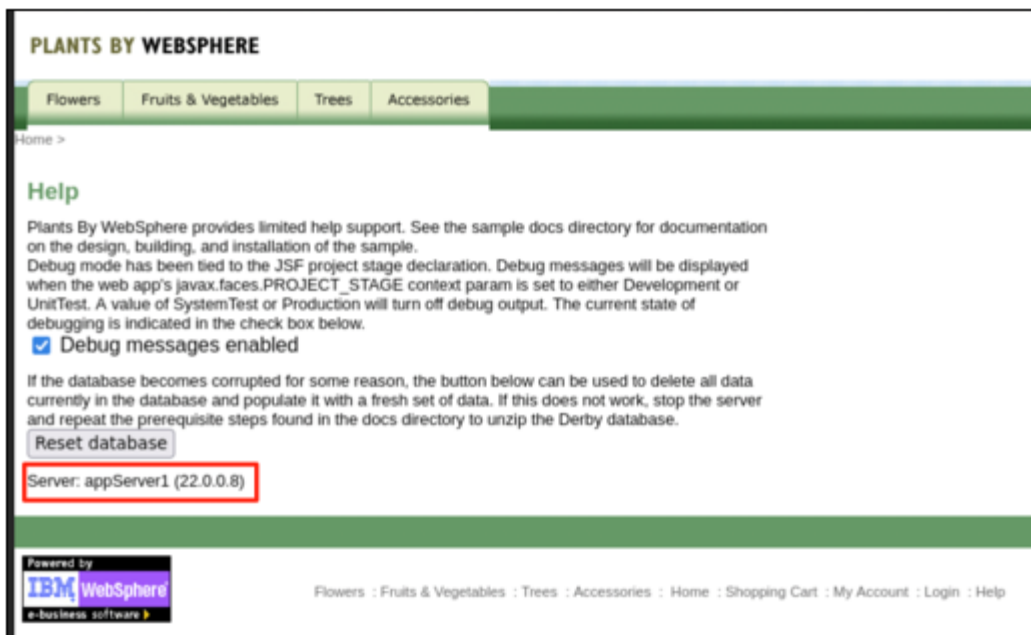
The server is stopped.



5. From the **PlantsByWebSphere** application page, click the "**Flowers**" tab, to show the catalog of flowers.



6. Click the **Help** link to go to application **Help** page, you can see now the application is running from a different application server.



This demonstrates that the Liberty dynamic routing detects the application server is down and directs the traffic to another application server automatically.

The application high availability test is completed.

## Test Case 2:

In this test case, the **WhereAml** application is used. This application does not use http sessions, and therefore the web server plugin can direct requests to the Liberty servers in a round-robin style.

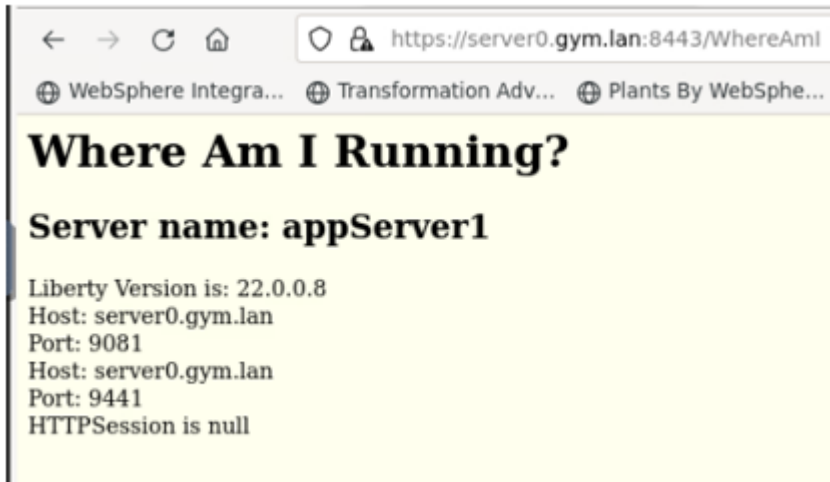
1. **Start** both application servers (appServer1 and appServer2) from Liberty collective Admin Center.
2. Open a new browser window and enter the **WhereAml** application URL as:

https://server0.gym.lan:8443/WhereAmI



The output shows that currently the application is running from **appServer1** server.

In your test, you may see **appSever2** handling the initial request.



3. Refresh the application page by clicking the **refresh icon** on the browser.

You can see the output showing that Liberty dynamic routing feature directs the request traffic to other application server in a round-robin fashion.



4. Refresh the browser again a few more times and see that the requests get routed to **appServer1** and **appServer2** accordingly.

The round robin load balancing test is completed.

## Part 6: Configure and Test Dynamic Routing Rules

With dynamic routing enabled, you can use routing rules in Liberty to customize exactly which servers are used to handle specific requests.

By default, dynamic routing balances load requests across all servers that can handle the request. To override the default behavior, you must configure routing rules. Routing rules can route requests to specific server resources, redirect requests, or reject requests.

Each **<routing Rules>** element can define the applicable set of **web servers** where the rules are published. In this example, there is only one web server, named **"webserver1"**.

When a web server connects to the DynamicRouting service, the service delivers rules to that web server.

### Example:

In this example, the routing rules will be applied to the **"webserver1"** HTTP server.

```
<dynamicRouting>
  <routing Rules webserver="webserver1">
    ...
  </routing Rules>
</dynamicRouting>
```

- The Web Server matches requests against the **match Expressions** that are found in the **<routingRule>**.
- The **<loadBalanceEndpoints>** defines the set of servers (**endpoint destinations**) that are permitted to handle the request based on the routing rules match expressions.

### Example:

In the example, requests that match **"/WhereAmI%"** will be routed to the Liberty server, **appServer1**.

```
<dynamicRouting>
  <routingRules webServers="webserver1">
    <routingRule order="100" matchExpression="URI LIKE '/WhereAmI%'">
      <permitAction>
        <loadBalanceEndPoints>
          <endpoint destination="server=*,*,*,appServer1"/>
        </loadBalanceEndPoints>
      </permitAction>
    </routingRule>
  </routingRules>
</dynamicRouting>
```

## IBM Documentation – Configuring routing rules from Dynamic Routing:

[https://www.ibm.com/docs/en/was-liberty/nd?topic=SSAW57\\_liberty/com.ibm.websphere.wlp.zseries.doc/ae/twlp\\_wve\\_routing\\_rules.htm](https://www.ibm.com/docs/en/was-liberty/nd?topic=SSAW57_liberty/com.ibm.websphere.wlp.zseries.doc/ae/twlp_wve_routing_rules.htm)

1. From the browser, go to the **WhereAmI** application URL again:

```
https://server0.gym.lan:8443/WhereAmI
```



2. Refresh the browser window for **WhereAmI** so you can see that the requests are getting routed to **appServer1** and **appServer2** accordingly as you observed in the previous section of the lab.
3. Run the following script, that is provided for the lab, to add the dynamic routing rules such that only appServer1 will handle the WhereAmI application requests.

```
~/liberty_admin_pot/lab-scripts/applyRoutingRules.sh -s appServer1
```



```
=====
Routing Rule is now set to send requests to 'appServer1' Liberty server(s)
SUCCESS: Routing Rules Applied.
=====

[techzone@server0 lab-scripts]$
```

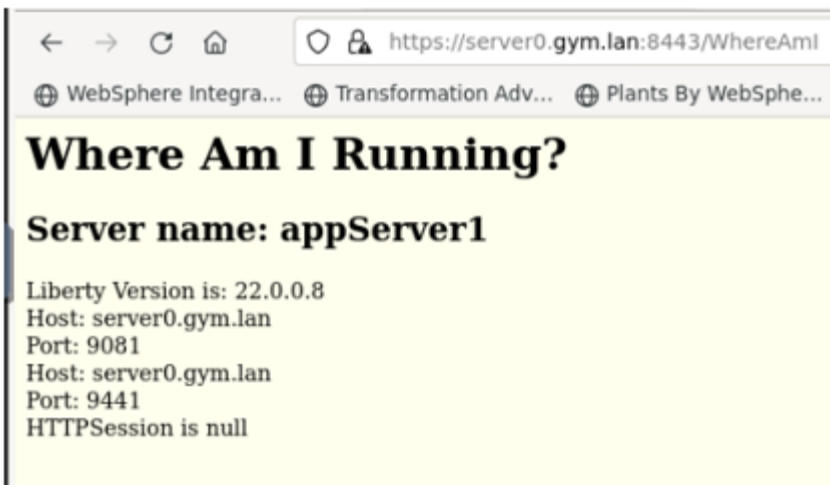


### Tip:

The dynamic rules are added to the Collective Controller by including the configuration in the server's **configDropins/overrides** directory.

Liberty dynamically applies the updated configuration to the controller.

4. Refresh the browser window for **WhereAmI** again and now you see that the requests are only routed to **appServer1**.



5. See the new routing rule illustrated below. The new routing rule states:

- The rules apply to the "webserver1" HTTP server
- The rule applies to requests that match "/WhereAmI" URI
- If the routing rule matches the expression, route the requests to Liberty server ONLY "appServer1"
- WhereAmI application will now only be routed to "appServer1"

```

1 <server description="Demonstrates Dynamic Routing Feature">
2   <featureManager>
3     <feature>dynamicRouting-1.0</feature>
4   </featureManager>
5
6   <dynamicRouting>
7     <routingRules webServers="webserver1">
8       <routingRule order="100" matchExpression="URI LIKE '/WhereAmI%'">
9         <permitAction>
10          <loadBalanceEndPoints>
11            <endpoint destination="server=*,*,*,appServer1"/>
12          </loadBalanceEndPoints>
13        </permitAction>
14      </routingRule>
15    </routingRules>
16  </dynamicRouting>
17
18 </server>

```

6. Run the following script again to change the destination to ONLY appServer2

```
~/liberty_admin_pot/lab-scripts/applyRoutingRules.sh -s appServer2
```



```
=====
Routing Rule is now set to send requests to 'appServer2' Liberty server(s)
SUCCESS: Routing Rules Applied.
=====

[techzone@server0 lab-scripts]$
```

7. Refresh the browser window for **WhereAmI** again and now you see that the requests are only routed to **appServer2**.

**Where Am I Running?**

**Server name: appServer2**

Liberty Version is: 22.0.0.8  
Host: server1.gym.lan  
Port: 9082  
HTTPSession is null

8. Run the following script to again to change the destination to **both** Liberty servers

```
~/liberty_admin_pot/lab-scripts/applyRoutingRules.sh -s all
```



```
=====
Routing Rule is now set to send requests to 'appServer1,appServer2' Liberty server(s)
SUCCESS: Routing Rules Applied.
=====

[techzone@server0 lab-scripts]$
```

9. Refresh the browser window for **WhereAmI** a few times. You will see that the requests get routed to **appServer1** and **appServer2** in a round-robin fashion.

## Summary

**Congratulations!**

**You have successfully completed the lab "The value of Dynamic Routing"**

In this Lab, you have seen how to configure the Dynamic Routing feature in Liberty collective to provide dynamic routing information to the web server plugin.

The ability to leverage Liberty's dynamic routing capability for any edition of Liberty is powerful and cost effective.

You get HA for **Liberty Base** and **Liberty Core** editions, without requiring Liberty ND licenses. It also eliminates time-consuming plug-in merges and manual updates to the configuration if servers are added or removed from the configuration.

This can also be very useful in topologies where servers or applications are regularly being added, changed, or removed.



